

I Built a Fake SSH Server on AWS - Then Watched the Internet Find It

A detailed build-and-investigate walkthrough: AWS → SSH → Cowrie → logs → analysis → evidence.

54

CONNECTION EVENTS

25

UNIQUE SOURCE IPs

40

COMMAND EVENTS

I wanted to understand every stage - not just run an installation command.

What is a honeypot, and what is its purpose?

A honeypot is a deliberately deployed decoy system or service designed to attract, observe and record suspicious or unauthorized interaction. It is used for deception, visibility, research and learning - not for storing real production data.

WHAT CAN BE A HONEYPOT?

SSH/Telnet decoys, web-application honeypots, malware-analysis sensors and multi-honeypot platforms can all serve different objectives.

MY FIRST CHOICE: T-POT

I initially tried T-Pot, a platform that combines multiple honeypot technologies. On my small cloud instance, Docker/resource problems made that design difficult to sustain.

THE PIVOT TO COWRIE

I kept the objective - observe SSH activity - but changed the tool. Cowrie was a lighter and more focused fit for the lab.

Changing the tool did not change the security question I wanted to answer.

What exactly did I deploy?

TECHNICAL DEFINITION

Cowrie is an SSH/Telnet honeypot that presents an interactive decoy environment and records authentication attempts, sessions, commands and related telemetry.

PLAIN-ENGLISH DEFINITION

It behaves like a fake Linux server. A visitor sees a believable shell, while I see what credentials were tried, what commands were entered and how the session behaved.

WHY COWRIE FIT THIS PROJECT

My goal was specifically to observe SSH behaviour. Cowrie worked within the available resources and gave me structured JSON logs that I could filter, correlate and investigate.

The key design principle: the fake service would face the Internet; my real administration path would remain protected.

Step 1: Log in to AWS and launch the Ubuntu server

This is the cloud computer that hosted the lab.

- 1 Sign in to the AWS Management Console and open EC2.
- 2 Choose Launch instance and give the lab instance a recognizable name.
- 3 Select Ubuntu Server as the operating system image.
- 4 Choose an instance size appropriate for the lab and select/create an SSH key pair.
- 5 Keep the downloaded `.pem`` private key safe - Windows will use it to authenticate to the real server.
- 6 Review storage/network settings, then launch the instance and wait for it to enter the Running state.

WHY THE `.PEM`` MATTERS

The key pair replaces a normal password for SSH authentication to the real EC2 host.

Step 2: Protect the real SSH management door

During instance setup, I restricted TCP/22 to my own public IP.

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.



Create security group



Select existing security group

We'll create a new security group called **'launch-wizard-2'** with the following rules:



Allow SSH traffic from

Helps you connect to your instance

My IP

1[REDACTED]/32



Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server



Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

WHAT THIS RULE MEANS

SSH uses TCP port 22. Setting the source to My IP means ordinary administrative SSH is not intentionally open to the whole Internet.

WHY I DID THIS

The experiment was about exposing the decoy, not exposing the real Ubuntu management service.

Step 3: Add a separate rule for Cowrie

I then added TCP/2222 as the Internet-facing honeypot port.

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

Security group rule ID	Type	Protocol	Port range	Source	Description - optional	
sgr-01234567	SSH	TCP	22	Custom	1.0.0.0/0	Delete
sgr-08765432	Custom TCP	TCP	2222	Custom	0.0.0.0/0	Delete

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

[Cancel](#) [Preview changes](#) [Save rules](#)

TWO DIFFERENT PATHS

TCP/22 → real Ubuntu administration, restricted.

TCP/2222 → Cowrie decoy, Internet-facing for observation.

IMPORTANT

A connection to port 2222 reaches Cowrie's simulated environment; it does not provide normal SSH access to the underlying Ubuntu host.

Step 4: Use the `.pem` key from Windows Command Prompt

After the EC2 instance was running, I connected to the real Ubuntu host over port 22.

cd %USERPROFILE%\Downloads

Move Command Prompt to the folder containing the downloaded key.

ssh -i "myhoneypotkey.pem" ubuntu@<PUBLIC-IP>

Authenticate with the private key and connect as the Ubuntu management user.

SUCCESSFUL SSH LOGIN TO THE REAL EC2 HOST - TCP PORT 22

```

The authenticity of host '[redacted]' can't be established.
[redacted] key fingerprint is [redacted].
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[redacted]' to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.6.0 [redacted] 6.64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Wed Sep  9 11:18:52 UTC 2026

System load: 0.08      Temperature: [redacted] -273.1 C
Usage of /:  7.2% of 27.95GB   Processes: [redacted]
Memory usage: 26%      Users logged in: 0
Swap usage:  0%          IPv4 address for ens5: [redacted]

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
  
```

WHAT I CHECKED

The Ubuntu/Debian login banner and real shell prompt confirmed that I was on the genuine EC2 host - not inside Cowrie.

Step 5: Confirm the tools needed for the build

Before installing Cowrie, I checked that Python and Git were available.

```
ubuntu@ip-10-10-10-10:~$ python3 --version
Python 3.8.10
ubuntu@ip-10-10-10-10:~$ git --version
git version 2.35.0
ubuntu@ip-10-10-10-10:~$ |
```

python3 --version

Python runs Cowrie and its dependencies.

git --version

Git is used to retrieve the Cowrie source repository.

Checking prerequisites first makes troubleshooting easier: if installation fails later, I already know the basic tools exist.

Step 6: Create a dedicated non-root Cowrie account

I did not want the Internet-facing application running as my administrator account.

```
ubuntu@ip-10-10-10-10:~$ sudo adduser --disabled-password cowrie
Changing the password for cowrie
Enter the new password (press ENTER for the default)
Full Name []:
Room Number []:
Work Phone Number (area and number) []:
Home Phone Number (area and number) []:
Other Phone Number (area and number) []:
Is the information correct? [Y/n] y
ubuntu@ip-10-10-10-10:~$
```

sudo adduser --disabled-password cowrie

Creates the service account without enabling a normal password login.

sudo su - cowrie

Switches into the dedicated Cowrie account for the installation.

SECURITY REASON

This follows least privilege: the service gets the access it needs without unnecessary administrative rights.

Step 7: Clone Cowrie with Git

Now I retrieved the project files under the dedicated service account.

```
cowrie@ip-10-0-1-10:~$ git clone https://github.com/cowrie/cowrie.git
Cloning into 'cowrie'...
remote: Enumerating objects: 26475, done.
remote: Counting objects: 100% (118/118), done.
remote: Compressing objects: 100% (111/111), done.
remote: Total 26475 (delta 63), reused 7 (delta 7), pack-reused 26357 (from 2)
Receiving objects: 100% (26475/26475), 12.97 MiB | 28.94 MiB/s, done.
Resolving deltas: 100% (18564/18564), done.
```

```
git clone https://github.com/cowrie/cowrie.git
```

Downloads the Cowrie source repository.

```
cd cowrie
```

Moves into the newly cloned project directory.

At this point I had the application source, but I still needed an isolated Python environment before installing dependencies.

Step 8: Isolate Cowrie's Python dependencies

A virtual environment gives the project its own Python package space.

```
cowrie@ip-10-10-10-10:~$ source cowrie-env/bin/activate
(cowrie-env) cowrie@ip-10-10-10-10:~$ which python
/home/cowrie/cowrie-env/bin/python
(cowrie-env) cowrie@ip-10-10-10-10:~$ python --version
Python 3.14.4
(cowrie-env) cowrie@ip-10-10-10-10:~$ |
```

python3 -m venv cowrie-env

Creates the virtual environment.

source cowrie-env/bin/activate

Activates it for the current shell.

which python && python --version

Confirms the shell is using the venv interpreter.

WHY THIS MATTERS

Project dependencies stay separate from the system-wide Python installation, reducing package conflicts.

Step 9: Install Cowrie and create its runtime structure

This turns the cloned source into a usable honeypot environment.

```
(cowrie-env) cowrie@i:~$ cowrie init
Wrote etc/cowrie.cfg
Created var/log/cowrie
Edit etc/cowrie.cfg to change defaults, then run 'cowrie start'.
(cowrie-env) cowrie@i:~$ ls
CHANGELOG.rst  LIC
CONTRIBUTING.rst  LIC
INSTALL.rst  MAN
(cowrie-env) cowrie@i:~$ pip show cowrie
Name: cowrie
Version: 3.0.14.dev22
Summary: Cowrie SSH/Telnet
Home-page: https://www.cowrie.org
Author:
Author-email: Michel C2006@hof.net
License-Expression: BSD
Location: /home/cowrie/.local/lib/python3.14/site-packages
Editable project location: /home/cowrie/.local/lib/python3.14/site-packages
Requires: attrs, bcrypt, cffi, idna, lark, packaging, pyasn1_modules, service_identity, treq, twisted
Required-by:
```

python -m pip install -e .

Installs Cowrie into the active virtual environment.

cowrie init

Initializes configuration, logs, downloads, TTY recordings and runtime directories.

Those directories later became the evidence sources I used during analysis and preservation.

Step 10: Put Cowrie on TCP/2222 - then prove it is listening

Configuration, process state and network state all needed to agree.

```
(cowrie-env) cowrie@ip-10-0-0-100 ~$ sed -n '2222' etc/cowrie.cfg
744:# (default: listen_
747:# For both IPv4 and
749:# e.g listen_endpoi
2
754:# the proxy's, e.g.
758:listen_endpoints =
(cowrie-env) cowrie@ip-10-0-0-100 ~$ sed -n '2222' etc/cowrie.cfg
proxy:tcp:2222:interface=0.0.0.0
.0
(cowrie-env) cowrie@ip-10-0-0-100 ~$ sed -n '2222' etc/cowrie.cfg
owrie start

Join the Cowrie communi
e.org/slack/

Starting cowrie: [twistd
var/run/cowrie.pid --logger cowrie.python.logfile.logger cowrie]...
(cowrie-env) cowrie@ip-10-0-0-100 ~$ sed -n '2222' etc/cowrie.cfg
owrie status
cowrie is running (PID:
(cowrie-env) cowrie@ip-10-0-0-100 ~$ ss -ltn | grep 2222
LISTEN 0      50          0.0.0.0:*
(cowrie-env) cowrie@ip-10-0-0-100 ~$ cat /etc/cowrie/cowrie.cfg
```

CONFIGURATION

`listen\_endpoints = tcp:2222:interface=0.0.0.0` tells Cowrie to listen on TCP/2222 on available IPv4 interfaces.

VERIFICATION

I started Cowrie, checked `cowrie status`, and used `ss -ltn | grep 2222` to confirm the operating system had a real listening socket.

Step 11: Test the fake server from Windows

Before analysing Internet traffic, I created a session that I knew was mine.

```
ssh -p 2222 admin1@<PUBLIC-IP>
```

Port 2222 deliberately connects to Cowrie instead of the real Ubuntu SSH service.

Inside the decoy I tried basic discovery commands:

```
whoami pwd ls cd /tmp
```

```
cat /etc/passwd cat /etc/hostname cat /etc/os-release
```

WHAT THIS TEST PROVED

The fake shell responded convincingly, and the backend recorded my session. That gave me a known baseline before unsolicited traffic arrived.

REAL VS FAKE

Port 22 = genuine EC2 filesystem. Port 2222 = Cowrie's simulated filesystem and decoy identities.

Step 12: Locate and read Cowrie's logs

After the test, I returned to the real host and inspected what Cowrie recorded.

KEY LOCATIONS

- ~/cowrie/var/log/cowrie/cowrie.json` → structured JSON
- ~/cowrie/var/log/cowrie/cowrie.log` → service log
- ~/cowrie/var/lib/cowrie/tty` → session recordings
- ~/cowrie/var/lib/cowrie/downloads` → captured files

```
cd ~/cowrie/var/log/cowrie
```

Move into the log directory.

```
cat cowrie.json
```

Read the active JSON event stream.

```
grep -h 'cowrie.command.input' cowrie.json*
```

Find command-entry events across active + rotated JSON logs.

```
grep -h 'cowrie.login.success' cowrie.json*
```

Find credentials Cowrie accepted inside the decoy.

The JSON file became my main structured source because I could filter individual event types and reconstruct sessions.

Step 13: Understand what each field is telling me

Raw telemetry becomes useful when every field answers an investigation question.

timestamp

When?

session

Which events belong together?

src_ip / src_port

Where did the connection appear to originate?

dst_port

Which service was reached?

eventid

Connect, login, command, close, etc.

username / password

Credentials presented to the decoy

input

Command entered in the fake shell

hashh

SSH negotiation fingerprint for comparison

This let me reconstruct one session from connection → authentication → command → disconnect.

[illegible]

IMPORTANT LANGUAGE

17

Step 15: Look for behaviour that repeats across sessions

The useful signal came from relationships between events.

```
/bin/./uname -s -v -n -r -m
```

Repeated system-discovery command: kernel, version/build, hostname, release and architecture.

THE PATTERN

Same source → separate short sessions → changing credentials → same discovery command. That was consistent with automated SSH credential scanning followed by reconnaissance.

ANOTHER KIND OF PROBE

Some connections sent `GET / HTTP/1.1` or `GET /favicon.ico HTTP/1.1` to the SSH port - behaviour consistent with automated service discovery.

ANALYST DISCIPLINE

I described what the evidence supported without claiming that an IP address identified a specific person, malware family or threat actor.

Step 16: Why did earlier activity seem to disappear?

The logs had not vanished - Cowrie had renamed older files by date.

```
(cowrie-env) cowrie@cowrie:~/var/log/cowrie$ ls -lah
total 248K
drwxrwxr-x 2 cowrie cowrie 4.0K Sep 12 01:40 .
drwxrwxr-x 3 cowrie cowrie 4.0K Sep 9 11:50 ..
-rw-rw-r-- 1 cowrie cowrie 0 Sep 9 11:50 .gitignore
-rw-rw-r-- 1 cowrie cowrie 9.1K Sep 12 07:48 cowrie.json
-rw-rw-r-- 1 cowrie cowrie 116K Sep 9 21:39 cowrie.json.2026-09-09
-rw-rw-r-- 1 cowrie cowrie 13K Sep 10 22:31 cowrie.json.2026-09-10
-rw-rw-r-- 1 cowrie cowrie 18K Sep 11 22:44 cowrie.json.2026-09-11
-rw-r--r-- 1 cowrie cowrie 2.6K Sep 12 07:48 cowrie.log
-rw-rw-r-- 1 cowrie cowrie 48K Sep 9 21:39 cowrie.log.2026-09-09
-rw-r--r-- 1 cowrie cowrie 4.0K Sep 10 22:31 cowrie.log.2026-09-10
-rw-r--r-- 1 cowrie cowrie 8.6K Sep 11 22:44 cowrie.log.2026-09-11
```

The active file remained `cowrie.json`, while previous days appeared as files such as `cowrie.json.2026-09-09`, `cowrie.json.2026-09-10` and `cowrie.json.2026-09-11`.

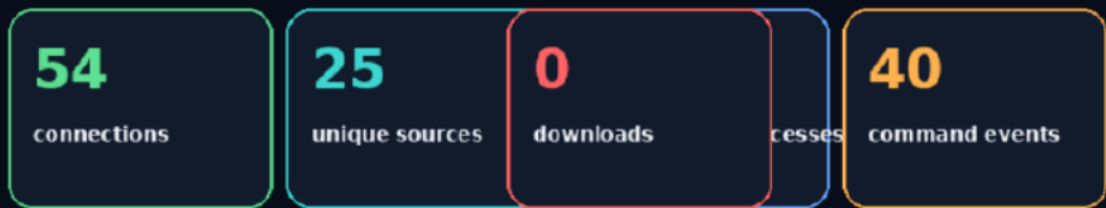
cowrie.json*

Using the wildcard included both the active file and rotated historical files in my searches.

SOC lesson: a correct query against an incomplete set of logs can still produce a misleading conclusion.

Step 17: Aggregate the complete multi-day telemetry

Once rotated logs were included, I counted the full dataset.



Counts are only a starting point. Some events were my controlled test, so I kept known test activity separate when interpreting unsolicited behaviour.

Step 18: Archive the logs before shutting the lab down

I preserved the material needed for later review.

```

}
(cowrie-env) cowrie/var/log/cowrie$ grep -h 'cowrie.session.connect' cowrie.json* \
| grep -o '"'
| sort -u \
| wc -l
25
(cowrie-env) cowrie/var/log/cowrie$ cd ~/cowrie

tar -czf ~/cowrie-evidence-2026-09-12.tar.gz \
var/log/cowrie \
var/lib/cowrie/tty \
var/lib/cowrie/downloads
(cowrie-env) cowrie@ip-172-31-17-56:~/cowrie$ ls -lh ~/cowrie-evidence-2026-09-12.tar.gz
-rw-rw-r-- 1 cowrie cowrie 29K Sep 12 09:59 /home/cowrie/cowrie-evidence-2026-09-12.tar.gz
(cowrie-env) cowrie@ip-172-31-17-56:~/cowrie$

```

```
tar -czf ~/cowrie-evidence-2026-09-12.tar.gz \
```

```
var/log/cowrie var/lib/cowrie/tty var/lib/cowrie/downloads
```

Logs + session recordings + download directory.

```
sha256sum ~/cowrie-evidence-2026-09-12.tar.gz
```

Generate a SHA-256 digital fingerprint of the archive.

The hash gave me an integrity value I could compare after transferring the evidence off the server.

Step 19: Copy the evidence to Windows and verify it

I used the genuine management channel - not the honeypot port.

```
ubuntu@ip-172-31-17-56:~$ ls -lh /home/cowrie/cowrie-evidence-2026-09-12.tar.gz /home/ubuntu/  
total 32K  
-rw-r--r-- 1 root root 29K Sep 12 10:57 cowrie-evidence-2026-09-12.tar.gz  
ubuntu@ip-172-31-17-56:~$
```

```
scp -i "%USERPROFILE%\Downloads\myhoneypotkey.pem" ubuntu@<PUBLIC-IP>:/home/ubuntu/c
```

Securely copy the archive from the real EC2 host.

Get-FileHash .\cowrie-evidence-2026-09-12.tar.gz -Algorithm SHA256

Calculate the Windows-side SHA-256 value.

VERIFY BEFORE SHUTDOWN

Matching hashes provide confidence that the transferred archive is byte-for-byte the same evidence package created on the server.

What began as 'install a honeypot' became an investigation

The value was in understanding why each step existed.

AWS login → EC2 → Security Group → PEM/SSH → Linux → least privilege → Git → Python venv → Cowrie → validation → JSON → correlation → log rotation → archive → SHA-256

- **What exactly does this log prove?**
- **Is this my controlled test or unsolicited traffic?**
- **Is this one event or a repeated pattern?**
- **Am I describing behaviour, or over-attributing identity?**

FULL STEP-BY-STEP TECHNICAL WRITE-UP

I will place the full documentation link in the LinkedIn post for the complete commands, screenshots, setup notes and analysis workflow.